Physics, Learning, or Both? Correcting a Sampling-Based MPC after an Unknown Payload Change

Shengwei Li
The University of Sydney, NSW, Australia
Project technical report, October 2026

***Abstract*—A robot arm that picks up an unknown payload makes the model inside its model-based controller wrong. The model can be corrected from a little motion data, either by estimating a physical parameter or by learning the residual with a neural network, or both. We compare these options in a deliberately small and fully controlled simulation study: a planar two-link arm driven by a sampling-based model predictive controller, three levels of model mismatch, an exact reference model, and a strict separation between development and test material. Three observations stand out. First, the amount of data decides the winner: with 9 s of motion, estimating the payload mass is more accurate than learning; with 30 s or more, learning wins inside the region where data was collected, and physics plus learning is best. Second, outside that region the controller exploits the learned model's errors; its one-step prediction error grows by an order of magnitude and most of the learned advantage disappears, with occasional failures of more than 50 cm. Third, averaging an ensemble of five networks removes many of these failures, but not when data is scarce, and at a computational cost beyond real time on the laptop used.**

I. Introduction

Model predictive control (MPC) chooses each action by predicting where candidate actions lead. Its decisions are only as good as its model. When a manipulator grasps an object of unknown mass, the model changes in a way the controller cannot see, and reaching accuracy drops.

There are two common ways to repair the model from data recorded after the change. One keeps the physics and estimates the unknown parameter, here the payload mass [1]. The other keeps the physics as a base and trains a neural network to predict what the base model gets wrong [2], [3]. Learned models are flexible, but they are only trustworthy where they have seen data, and an MPC actively searches for actions that its model rates highly. This report asks a narrow question: *in this setting, when does a learned correction help a sampling-based MPC, and when does it hurt?*

We answer it in a setting chosen for control over every source of error, not for realism (Sec. III). The arm is simple enough that the controller's model can be written by hand and verified against the simulator term by term, so that an exact reference model exists and every remaining error can be attributed. Within this setting we observe:

1) With little data, estimating the payload mass beats learning; with 30 s or more, learning wins inside the training region, and physics plus learning is best.
2) Outside the training region, the controller exploits learned-model errors: one-step prediction errors along the executed path grow 8–13 times, and the learned advantage largely disappears.
3) Every wrong model is exploited in the same way: the controller's expected outcome is optimistic by half to all of the model's prediction error. Learned models outside their data simply give it more error to exploit.
4) Averaging five networks reduces failures and makes physics plus learning the most robust option outside the training region given enough data; a penalty on ensemble disagreement added nothing measurable.

II. Related Work

Rigid-body dynamics are linear in the inertial parameters, so payload and link parameters can be estimated by least squares from joint motion and torque [1]. Learning a residual on top of a nominal physical model has been used for drone landing [2] and, with a Gaussian-process error model inside MPC, for trajectory tracking with a robot arm [3]; [4] reviews learning-based MPC. In model-based reinforcement learning, neural-network dynamics are combined with sampling-based MPC [5], and ensembles of probabilistic models are used to represent and propagate uncertainty [6]. Path-integral control [7] has been extended to account for uncertain physical parameters [8]. This report does not propose a new method; it compares these ingredients in one controlled setting and looks at how they fail.

III. Setup

A. Arm and worlds

The simulated arm (MuJoCo [9], 2 ms step) has two 0.3 m links of 1 kg, modelled as uniform rods, moving in a vertical plane under gravity, with torque limits of 20 and 10 N m at shoulder and elbow (Fig. 1). All experiments use a 1 kg payload. The real arm can differ from the controller's model in three ways, called worlds:

- **W1**: only the payload mass is unknown; the payload is a point mass at the tip.
- **W2**: the payload's centre of mass is offset from the tip by 4 cm along and 6 cm across the second link.
- **W3**: as W2, plus joint friction that the model lacks: extra viscous friction (0.3 and 0.15 N m s/rad) and Coulomb friction (0.6 and 0.3 N m, smoothed below 0.2 rad/s).

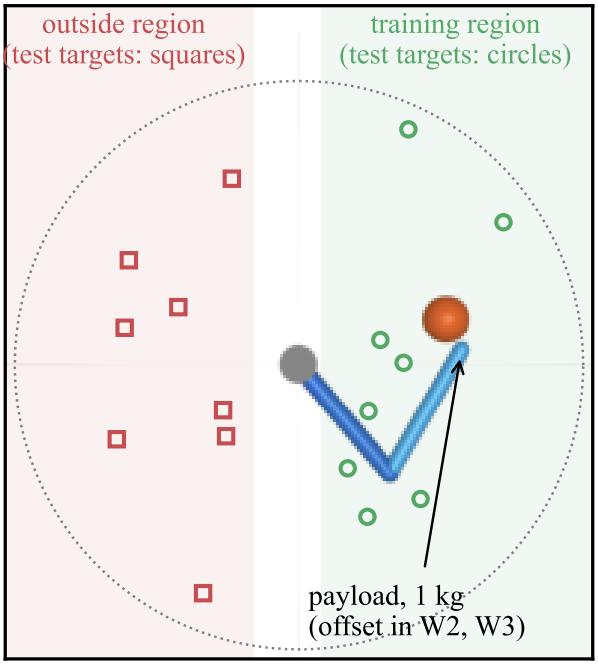


Fig. 1. The arm with its payload, the region where training data was collected (right, $x \geq 0.05$ m) and the region it never aimed at (left, $x \leq -0.1$ m), with the targets of the 8 test tasks in each.

In W1 a correct payload mass makes the model exact; in W2 and W3 no single mass does.

B. Controller

All methods share one sampling-based MPC with path-integral weighting [7]. Every 20 ms it perturbs its current torque plan to form 256 candidates over a 0.2 s horizon, predicts each candidate with the method's model, scores it by the squared tip-to-target distance plus a joint-speed penalty, and blends the candidates with weights that decrease exponentially with cost. Only the first torque of the blended plan is applied. The settings were tuned once, on development tasks, and then fixed for every method; only the predictor changes.

C. Methods

Table I lists the predictors. The hand-written model of the arm, including the offset payload and friction terms needed by the reference, was checked against the simulator's inertia matrix and bias forces and reproduces the simulator step for step when given the true parameters. C1 fits the payload mass by Gauss–Newton least squares on the one-period velocity prediction error. The networks in C2 and C3 take the sines and cosines of the joint angles, the joint speeds and the torques, and predict the base model's error in angles and speeds after one 20 ms period. They have two hidden layers of 64 tanh units and are trained with Adam and early stopping on held-out episodes. The ensembles add four networks that differ only in their random initialisation.

D. Data and protocol

Data is recorded by running C0 with random torque perturbations towards random targets in the training region, in 3 s episodes, and is measured in seconds of motion: budgets of 9, 30 and 90 s, with smaller budgets being the first episodes of the largest. Data is always split by whole episodes.

TABLE I
METHODS. ALL USE THE SAME MPC; ONLY THE PREDICTOR DIFFERS.

C0	physics model assuming no payload
C1	physics model with the payload mass estimated from data
C2	C0's model plus a residual network
C3	C1's model plus a residual network
C2-E, C3-E	as C2, C3, with the average of five networks; optionally the MPC is penalised for the networks' disagreement
Reference	physics model told the true payload and world (an upper bound, not a method)

Every choice (MPC settings, network size and training, penalty weight) was made on development material only: a separate data seed, a separate validation set and separate development tasks. The test material consists of three further data seeds and 8 reaching tasks inside and 8 outside the training region (Fig. 1), giving 24 runs per method, world and budget. A run lasts 3 s; its final error is the mean tip error over the last 0.5 s, and it counts as not settled if the error at the end exceeds 1 cm. Re-running the inside tasks in a later experiment reproduced all 792 earlier runs exactly.

IV. RESULTS

A. Prediction

Fig. 2 shows how far each model's prediction of the tip drifts in 0.2 s, the MPC's horizon. C1 is exact in W1 with any amount of data, but in W2 and W3 it leaves an error of about 2.5–4 cm that more data does not reduce: it can only explain the offset payload and friction by overestimating the mass (about 1.2 kg). C2 needs data: with 9 s it is far worse than C1, with 90 s far better. C3 is the most accurate throughout, because its network only has to learn what C1 leaves. Outside the training region the learned models degrade by roughly an order of magnitude, while C1 does not.

B. Control inside the training region

Fig. 3 (solid lines) and Table II show control results. C0 stops 8–10 cm from the target. C1 settles about 2 cm away in W2 and W3, consistently, with almost no spread. With 30 s of data or more, the learned methods do better; with 90 s, C3 reaches 0.07 cm in W2, close to the reference (0.04 cm). With 9 s the picture reverses: C2 is worse than C1, and in W2 one run ends 68 cm from the target. The two kinds of method fail differently: C1 is consistently a little off, the learned methods are usually precise but occasionally far off.

C. Control outside the training region

Outside the training region (dashed lines in Fig. 3), C1 performs as it does inside, but the learned methods lose most of their advantage. In W2 with 90 s of data, C2 goes from 0.36 cm inside to 1.78 cm outside, no better than C1; with 30 s it is twice as bad as C1. C3 holds up better, staying ahead of C1 with 90 s

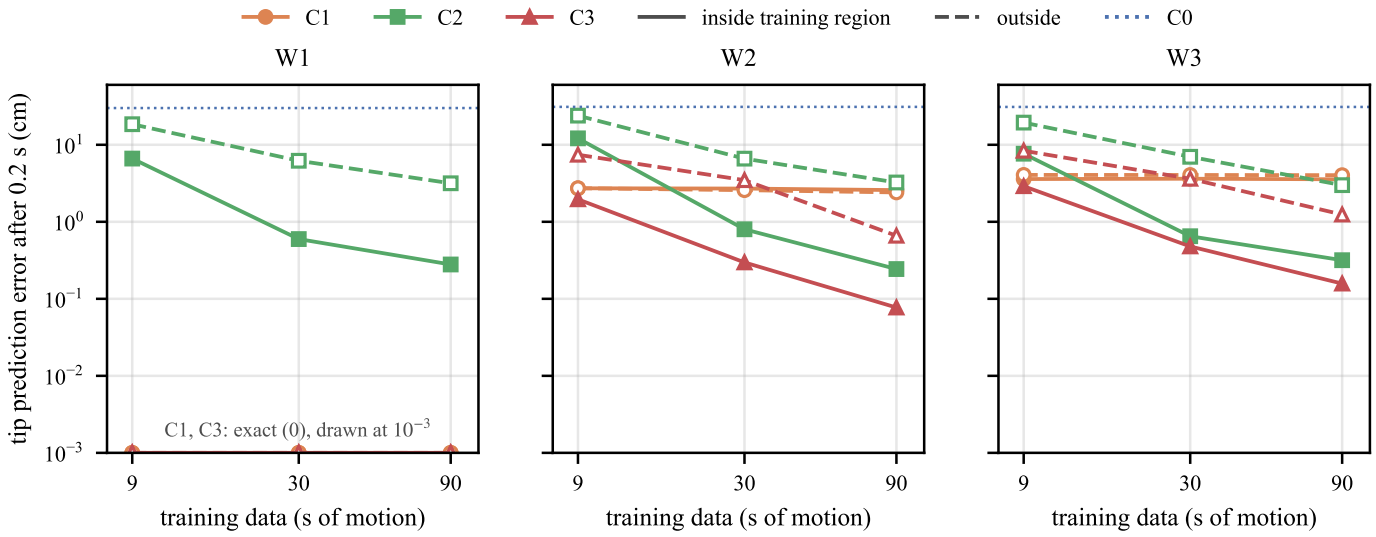


Fig. 2. Tip prediction error after 0.2 s against the amount of training data (mean of three data seeds), on validation episodes inside (filled, solid) and outside (open, dashed) the training region. The reference model is exact everywhere.

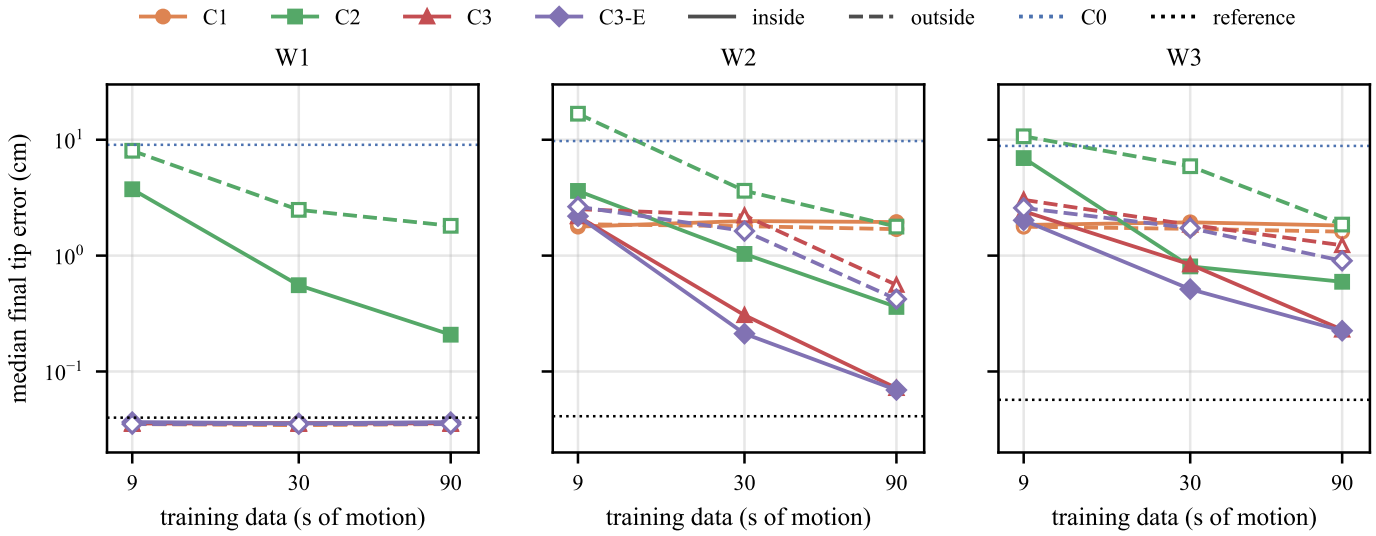


Fig. 3. Median final tip error over 24 runs (8 tasks, 3 data seeds) against the amount of training data, for test tasks inside (solid) and outside (dashed) the training region. Dotted lines: C0 and the reference.

(0.56 cm against 1.69 cm) but falling behind it with 30 s. Fig. 4 shows the three worst runs: on tasks that C1 solves directly, C2 swings the arm through large loops and ends 49–66 cm away.

D. Why: the controller's expectations are optimistic

Every control round, the MPC predicts where the tip will be one period later under the torque it applies. Comparing that prediction with where the tip actually goes measures the model's error along the path the controller chose. For the learned models it grows 8–13 times outside the training region (C2, W2, 30 s: from 0.0095 to 0.079 cm per period), while for C1 it stays at 0.03 cm.

The error is not random in direction. We define optimism as how much closer to the target the model expected the tip to be than it actually was. Random errors would average out near zero; instead, for every wrong model, physical or learned, inside or outside, optimism amounts to half to all of the prediction error (Fig. 5). This is a selection effect: by picking the action its model rates best, the MPC systematically picks actions whose outcome the model overestimates. What sets the learned models apart outside their data is not this mechanism but the size of the errors it acts on.

E. Ensembles

Averaging five networks helps mostly with the tails (Fig. 6, Table II). Outside the training region in W2 with 90 s of data, the number of unsettled runs of physics plus learning drops from 6 to 2; with 30 s, C3-E (1.63 cm) is no longer worse than C1

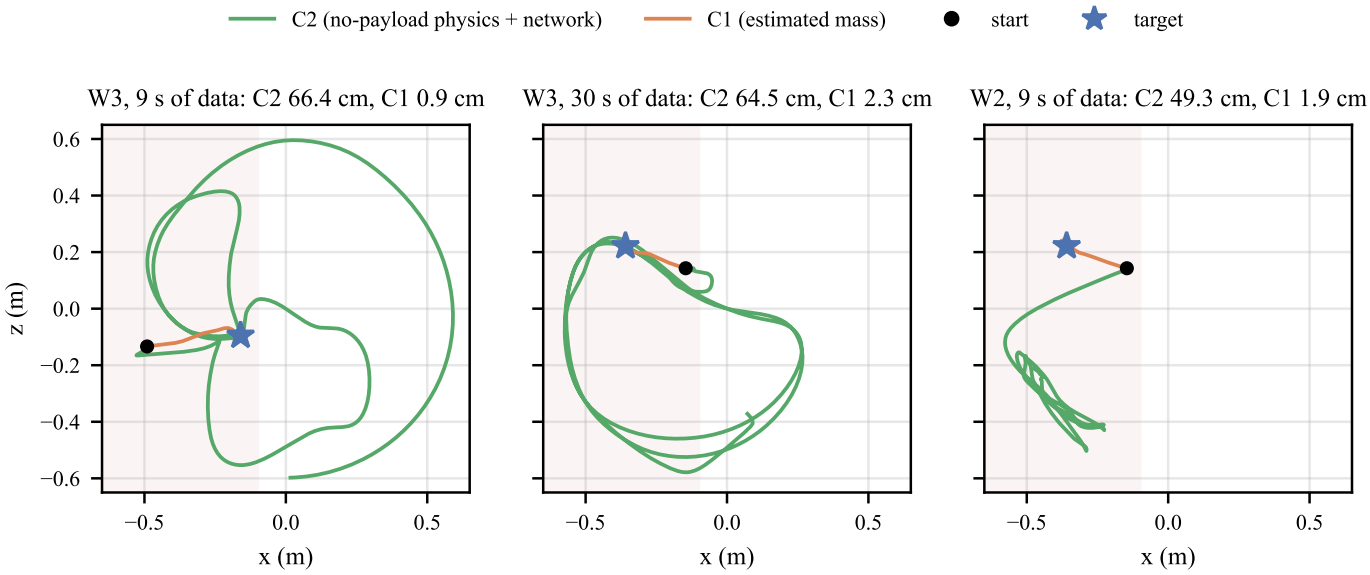


Fig. 4. Tip paths of the three worst C2 runs outside the training region (shaded), next to C1 on the same task, data seed and MPC seed.

TABLE II
MEDIAN FINAL TIP ERROR IN CM (RUNS NOT SETTLED, OF 24). C3-E AVERAGES FIVE NETWORKS WITHOUT PENALTY.

Tasks	World	Data	C1	C2	C3	C3-E	Ref.
inside	W2	30 s	1.99 (24)	1.03 (11)	0.31 (4)	0.21 (2)	0.04 (0)
inside	W2	90 s	1.95 (22)	0.36 (4)	0.07 (1)	0.07 (0)	0.04 (0)
inside	W3	30 s	1.94 (24)	0.81 (9)	0.84 (8)	0.51 (6)	0.06 (0)
inside	W3	90 s	1.82 (22)	0.59 (6)	0.23 (4)	0.23 (3)	0.06 (0)
outside	W2	30 s	1.79 (20)	3.63 (22)	2.22 (21)	1.63 (17)	0.04 (0)
outside	W2	90 s	1.69 (19)	1.78 (18)	0.56 (6)	0.42 (2)	0.04 (0)
outside	W3	30 s	1.70 (18)	5.91 (24)	1.84 (18)	1.73 (20)	0.06 (0)
outside	W3	90 s	1.61 (17)	1.85 (21)	1.23 (15)	0.90 (10)	0.06 (0)

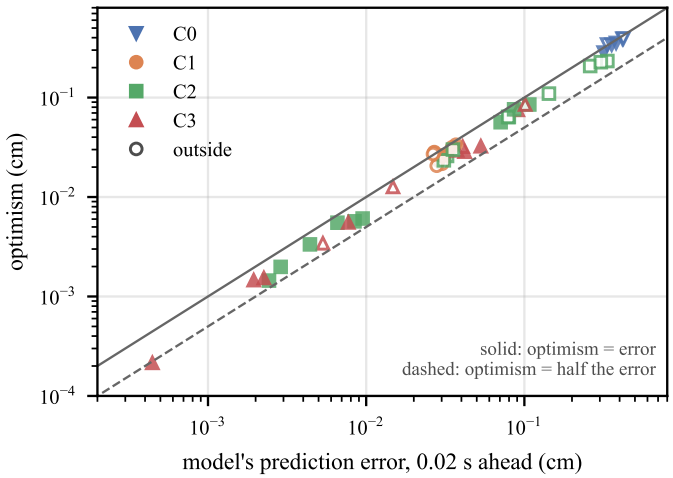


Fig. 5. Median optimism against median one-period prediction error along the executed path, one point per method, world, data budget and task region (exact models omitted). All points lie between half and all of the error.

(1.79 cm), unlike the single-network C3. With 9 s of data, the ensemble does not help: all five networks are poor, and C2-E remains far worse than C1 outside the training region.

Penalising disagreement between the networks was meant to keep the MPC away from states the data did not cover. On development tasks, penalty weights from 0.3 to 3000 changed the error by little, and on test tasks the penalised ensembles were neither consistently better nor worse than plain averaging. A likely reason is that disagreement was measured one step ahead from the averaged state, where it is small, whereas errors build up over the horizon; propagating each network separately, as in [6], would capture this at a higher cost.

Cost is the other limit. Planning one round took about 6 ms for C1, 11 ms for C3 and 28 ms for C3-E on the laptop CPU used, so the ensemble exceeds the 20 ms control period. The simulation waits for the controller, so this does not affect the results above, but the ensemble controller could not run in real time as implemented.

V. Discussion and Limitations

Read practically, the results suggest: with little data, estimate the physical parameter; with more, add a learned residual on top of the physics and average several networks; and judge a learned controller by its worst runs and by where it will be used, not only by its median error inside the data it was trained on.

The study is deliberately narrow. It is simulation only, with a planar two-link arm and one payload mass; “outside” is one region of the workspace; the ensemble members differ only in initialisation; and the model mismatches in W2 and W3 were chosen by hand to be large enough to matter. The disagreement penalty was a simple one-step version and should not be read as evidence against uncertainty-aware planning in general.

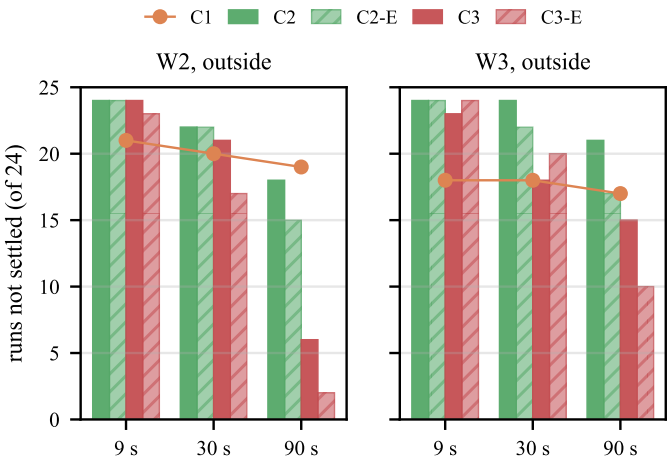


Fig. 6. Runs not settled (of 24) outside the training region: single networks (dark) against five-network averages (light), with C1 for reference.

VI. Conclusion

In a controlled setting where every source of model error is known, a learned correction made a sampling-based MPC markedly more accurate when enough data was available and the task stayed where the data was collected, and markedly less reliable when either condition failed. The mechanism is that the controller exploits whatever errors its model has, and learned models outside their data have large ones. Physics plus an averaged ensemble was the most robust combination we tested, at a computational cost that needs to be addressed before it can run in real time.

References

[1] C. G. Atkeson, C. H. An, and J. M. Hollerbach, "Estimation of inertial parameters of manipulator loads and links," *The International Journal of Robotics Research*, vol. 5, no. 3, pp. 101–119, 1986.

[2] G. Shi, X. Shi, M. O'Connell, R. Yu, K. Azizzadenesheli, A. Anandkumar, Y. Yue, and S.-J. Chung, "Neural lander: Stable drone landing control using learned dynamics," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2019, pp. 9784–9790.

[3] A. Carron, E. Arcari, M. Wermelinger, L. Hewing, M. Hutter, and M. N. Zeilinger, "Data-driven model predictive control for trajectory tracking with a robotic arm," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 3758–3765, 2019.

[4] L. Hewing, K. P. Wabersich, M. Menner, and M. N. Zeilinger, "Learning-based model predictive control: Toward safe learning in control," *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 3, pp. 269–296, 2020.

[5] A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine, "Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2018, pp. 7559–7566.

[6] K. Chua, R. Calandra, R. McAllister, and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," in *Advances in Neural Information Processing Systems*, vol. 31, 2018.

[7] G. Williams, A. Aldrich, and E. A. Theodorou, "Model predictive path integral control: From theory to parallel computation," *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017.

[8] I. Abraham, A. Handa, N. Ratliff, K. Lowrey, T. D. Murphey, and D. Fox, "Model-based generalization under parameter uncertainty using path integral control," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 2864–2871, 2020.

[9] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.